

COMP 2402

Hash Tables

part 2

Universal Hashing

We select a hash function **at random** from a family of hash functions with a certain mathematical property. This guarantees a low number of collisions in expectation.

We want for any $x, y \in \{0, \dots, 2^w - 1\}$, $x \neq y$ $Pr(\text{hash}(x) = \text{hash}(y)) \leq \frac{1}{t.\text{length}}$

Example of a family of hash functions:

Let z be a random number in $\{0, \dots, 2^w - 1\}$, and let $t.\text{length}$ be a prime number.

Then the formula for hashing an integer x is

Carter & Wegman - 1979



$$\text{hash}(x) = (z \cdot x) \bmod t.\text{length}$$

expensive

Family of hash functions

$$\text{hash}(x) = (z \cdot x) \bmod t.\text{length}$$

Improvement: let $t.\text{length}$ be 2^k (for some integer k)

$$y \bmod 2^k \equiv \text{last } k \text{ bits of } y$$

Problem: there are certain sets of integers that are hashed with lots of collisions. With this hash function you can get lists of logarithmic length.

Multiplicative Hashing

Dietzfelbinger et al. - 1997

$$\text{hash}(x) = ((z \cdot x) \bmod 2^w) \operatorname{div} 2^{w-d}$$

computer code: $z * x \gg (w - d)$

right shift by $w - d$

- In multiplicative hashing, we use a hash table of size 2^d for some integer d (called the **dimension**): **$t.\text{length} = 2^d$** .
- x is an integer we want to hash: $x \in \{0, \dots, 2^w - 1\}$
- z is a randomly chosen **odd** integer: $z \in \{1, 3, 5, \dots, 2^w - 1\}$
- **div** operator = integer division (we discard the remainder):
for any integers $a \geq 0, b \geq 1$, $a \operatorname{div} b = \lfloor a/b \rfloor$
- Computers with **32-bit** arithmetic keep only the **lowest 32** bits of any number. So, by default, operations on integers are already done modulo 2^w .

Review

Binary multiplication:

$$\begin{array}{r}
 \times 1001 \\
 100 \quad \leftarrow 2^2 \\
 \hline
 0000 \\
 + 0000 \\
 1001 \\
 \hline
 100100
 \end{array}$$

$$2^w = \underbrace{1000000 \dots 0000}_{w \text{ bits}}$$

$$2^w - 1 = \underbrace{111111 \dots 1111}_{w \text{ bits}}$$

$$x \operatorname{div} 2^3 = 1001110 \quad \leftarrow x \gg 3 \quad \text{right shift by 3}$$

$$x = 1001110101$$

$$2^8 = 1 \text{ } \underbrace{00000000}_{\text{pink}}$$

$$x \cdot 2^8 = 1001110101 \text{ } \underbrace{00000000}_{\text{pink}} \quad \leftarrow x \ll 8$$

left shift by 8

$$x = 1001110101$$

$$x \operatorname{mod} 2^7 = 1110101$$

Example from the textbook

$$\text{hash}(x) = ((z \cdot x) \bmod 2^w) \operatorname{div} 2^{w-d}$$

$w = 32$

$$d = 8$$

[illegible]

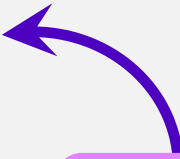
Multiplicative Hashing

$$\text{hash}(x) = ((z \cdot x) \bmod 2^w) \operatorname{div} 2^{w-d}$$

For any randomly chosen **odd** integer $z \in \{1, 3, 5, \dots, 2^w - 1\}$,
and for any $x, y \in \{0, \dots, 2^w - 1\}$, $x \neq y$,

$$\Pr(\text{hash}(x) = \text{hash}(y)) \leq \frac{2}{2^d}$$

$t.\text{length}$



Theorem 5.1

A **ChainedHashTable** implements the **USet** interface. Ignoring the cost of calls to `resize()`, a **ChainedHashTable** supports the operations `add( $x$ )`, `remove( $x$ )`, and `contains( $x$ )` in $O(1)$ **expected** time per operation.

Furthermore, beginning with an empty **ChainedHashTable**, any sequence of m `add( $x$ )` and `remove( $x$ )` operations results in a total of $O(m)$ time spent during all calls to `resize()`.

Hash Codes

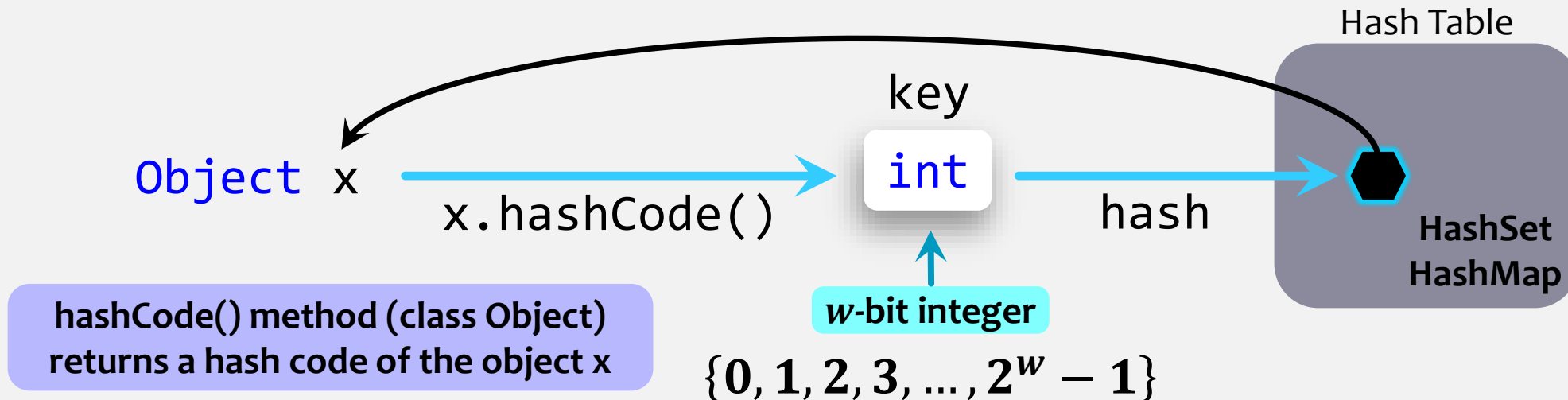
Very often hash tables store types of data that are **not integers**. They may be strings, objects, arrays, or other compound structures.

We must **map** these data types to **w-bit** hash codes:

`hashCode()` returns a **w-bit** integer in $\{0, \dots, 2^w - 1\}$.

1. **Require:** if `x.equals(y)` then `x.hashCode() == y.hashCode()`
2. **Desire:** if `!x.equals(y)` then `x.hashCode() != y.hashCode()`

$$\Pr(x.\text{hashCode()} == y.\text{hashCode}()) \leq \frac{1}{2^w}$$



Hashing & Java Data Types

DATA TYPE	SIZE	DESCRIPTION
byte	8 bits	Stores whole numbers from –128 to 127
char	16 bits	Stores a single character/letter or ASCII values
short	16 bits	Stores whole numbers from –32,768 to 32,767
int	32 bits	Stores whole numbers from –2,147,483,648 to 2,147,483,647
float	32 bits	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
long	64 bits	Stores whole numbers from –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
double	64 bits	Stores fractional numbers. Sufficient for storing 15 decimal digits

Hashing & Java Data Types

Easy
treat these bits as
the representation
of an **integer** in
the range
 $\{0, \dots, 2^{32} - 1\}$

Difficult
Chapter 5.3.2

WRAPPER CLASSES	DATA TYPE	SIZE	BINARY REPRESENTATION
Byte	byte	8 bits	$\{0, \dots, 2^8 - 1\} = \{0, \dots, 255\}$
Character	char	16 bits	$\{0, \dots, 2^{16} - 1\} = \{0, \dots, 65535\}$
Short	short	16 bits	$\{0, \dots, 2^{16} - 1\} = \{0, \dots, 65535\}$
Integer	int	32 bits	$\{0, \dots, 2^{32} - 1\} = \{0, \dots, 4,294,967,296\}$
Float	float	32 bits	$\{0, \dots, 2^{32} - 1\} = \{0, \dots, 4,294,967,296\}$
Long	long	64 bits	$\{0, \dots, 2^{64} - 1\}$
Double	double	64 bits	$\{0, \dots, 2^{64} - 1\}$

Hashing & Java Data Types

- Class Long

```
public int hashCode()  
    return (int)(this.longValue()^(this.longValue()>>>32))
```

Exclusive OR (XOR)

p	q	$p \oplus q$
1	1	0
1	0	1
0	1	1
0	0	0

- Class Double

```
public int hashCode()  
    long v = Double.doubleToLongBits(this.doubleValue());  
    return (int)(v^(v>>>32));
```

This is bad
when first 32 bits are
the same as last 32 bits

(unsigned)
bitwise right shift operator

Hashing & Java Data Types

- Interface Map.Entry<K,V>

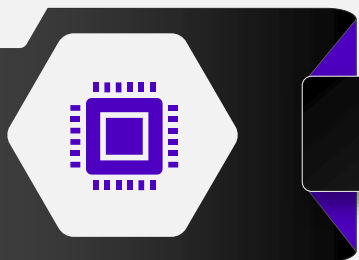
```
boolean equals(Object o)
```

```
    return (e1.getKey()==null ?  
            e2.getKey()==null : e1.getKey().equals(e2.getKey())) &&  
            (e1.getValue()==null ?  
            e2.getValue()==null : e1.getValue().equals(e2.getValue()))
```

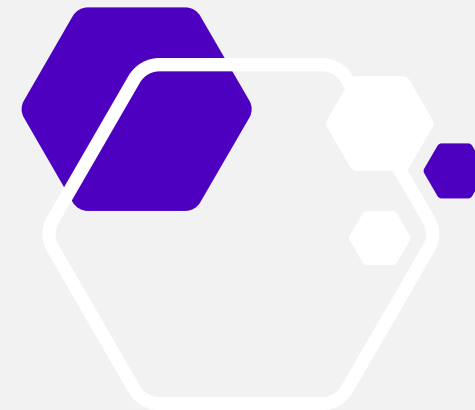
two entries are the same if their keys are the same and their values are the same

```
int hashCode()
```

```
    return (e.getKey()==null ? 0 : e.getKey().hashCode()) ^  
            (e.getValue()==null ? 0 : e.getValue().hashCode())
```



BadExamples.java



```
long y = 1L << 32L; // y = 2^32 = 4294967296
```

HashSet is just a HashMap in Java:

<http://hg.openjdk.java.net/jdk8/jdk8/jdk/file/687fd7c7986d/src/share/classes/java/util/HashSet.java>

Fix with BSTs:

<http://hg.openjdk.java.net/jdk8/jdk8/jdk/file/687fd7c7986d/src/share/classes/java/util/HashSet.java>

```
public class BadExamples {
    static class Point {
        int x, y;
        Point(int x, int y) {
            this.x = x;
            this.y = y;
        }

        public boolean equals(Object o) {
            if (o == this) {
                return true;
            }
            if (!(o instanceof Point)) {
                return false;
            }
            Point c = (Point) o;
            return c.x == x && c.y == y;
        }

        public int hashCode() {
            return x ^ y
        }
    }
}
```

Good Hash Code

For a compound object, we want to create a hash code by combining the individual hash codes of the object's constituent parts.

hardcoded →

Change this every time you restart your app, otherwise someone can design an attack.

```
public int hashCode() {  
    // random numbers from rand.org  
    long[] z = {0x2058cc50L, 0xcb19137eL};  
    long zz = 0xbea0107e5067d19dL;  
  
    // convert (unsigned) hashcodes to long  
    long h0 = x & ((1L<<32)-1);  
    long h1 = y & ((1L<<32)-1);  
  
    return (int)((((z[0]*h0 + z[1]*h1)*zz) >>> 32));  
}
```

Hash Codes for Compound Objects

Given an object made up of $r \geq 1$ parts whose hash codes are $x_0, x_1, \dots, x_{r-1}$

- choose mutually independent random w -bit integers $z_0, z_1, \dots, z_{r-1}$
- choose a random $2w$ -bit **odd** integer z
- compute a hash code for our object:

$$\text{hash}(x) = \text{hash}(x_0, \dots, x_{r-1}) = \left(\left(z \cdot \sum_{i=0}^{r-1} z_i x_i \right) \bmod 2^{2w} \right) \text{div } 2^w$$

Given different objects x and y , assume $x_i \neq y_i$ for at least one index $i \in \{0, \dots, r-1\}$.

Then

$$\Pr(\text{hash}(x_0, \dots, x_{r-1}) = \text{hash}(y_0, \dots, y_{r-1})) \leq \frac{3}{2^w}$$